

Glen Olinger - GEOG485 - 25 July 2026

Columbia River Bar Channel Analysis Scripts Descriptions

The Python scripts in this project form a tool that executes an automated seafloor change-analysis workflow. The analysis is designed to understand how the seafloor changes from erosion and deposition perspectives. The target area is the Columbia River Bar channel, which is maintained by the U.S. Army Corps of Engineers to support deep-draft vessel passage across the bar between the Pacific Ocean and the Columbia River.

Download New Surveys

Purpose

The `download_new_surveys_v4.py` script queries the eHydro survey service for survey records in a user-defined bounding box, filters the records to the Columbia River Bar surveys, and downloads only surveys that have not already been downloaded.

Function Descriptions

1. `parse_yyyymmdd(value: str) -> date`

This helper function converts a string in `YYYYmmdd` format into a Python `date` object. It is used to define start and end date parameters for the survey date filter.

2. `extract_survey_date(survey_id: str) -> date | None`

This helper function extracts an 8-digit date token from a survey ID string using a regular expression pattern. If a valid token is found, it is converted to a `date`; otherwise, `None` is returned.

3. `query_surveys_page(...) -> list[dict]`

This function sends one paged request to the eHydro REST service. It applies:

- a bounding box geometry filter,
- a text filter on survey ID and survey name,
- sorting by survey ID,
- and pagination controls (`resultRecordCount` and `resultOffset`).

It returns a list of survey feature records from the service response.

4. `download_file(url: str, destination: Path) -> None`

This function performs the file download for one survey ZIP file. It streams the response from the source URL and writes it to the output path.

5. `load_downloaded_ids(outdir: Path) -> set[str]`

This function scans the output directory for existing ZIP files and builds a set of downloaded survey IDs (uppercase filename stems). This set is used to prevent re-downloading previously downloaded surveys.

6. `check_and_download_surveys(...) -> bool`

This is the primary workflow function for the script. It:

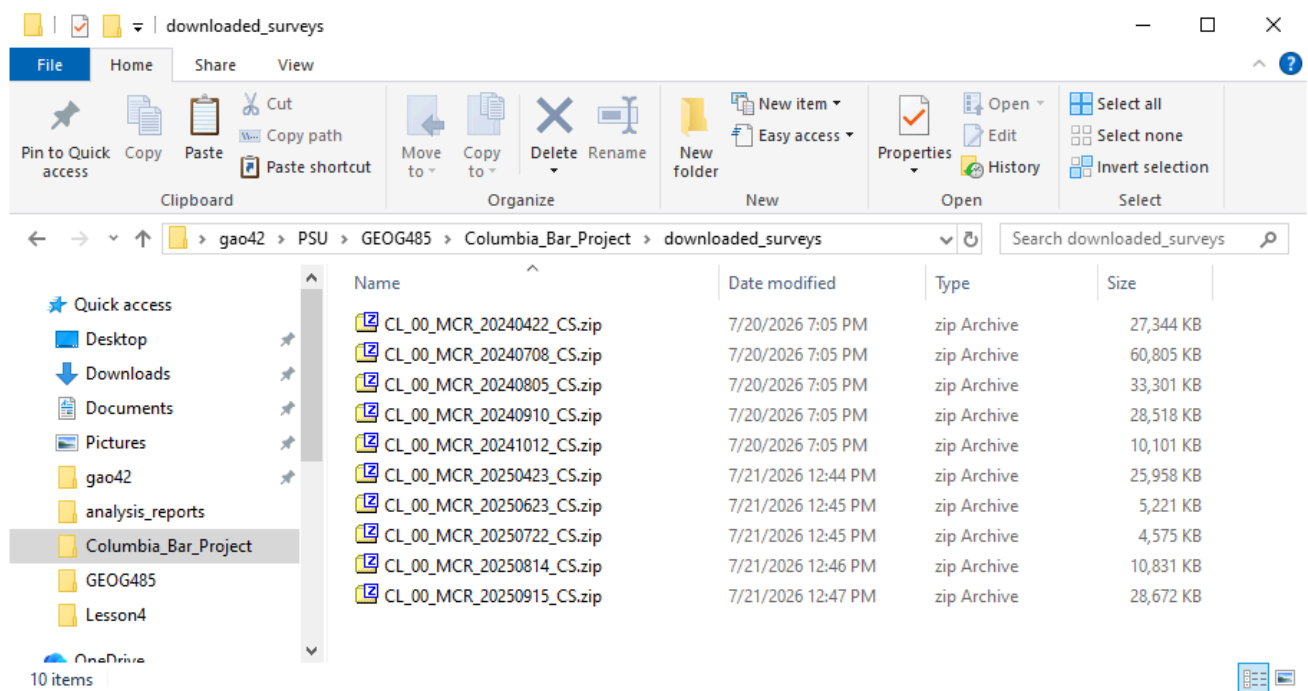
- validates runtime parameters,
- ensures the output directory exists,
- loads known downloaded surveys,
- queries the survey service page-by-page,
- applies ID/name filtering and date-window filtering,
- skips invalid, duplicate, or already-downloaded records,
- downloads new survey ZIP files,
- and reports summary counts.

It returns **True** if at least one new survey was downloaded, and **False** if none were downloaded.

```
7. main() -> int
```

This function sets the parameters for the script, calls `check_and_download_surveys(...)` for unit testing & development and prints a final message indicating whether any new surveys were downloaded.

Downloaded Surveys Directory



Preprocess Surveys

Purpose

The `preprocess_surveys_v2.py` script preprocesses downloaded survey ZIP files by extracting only archives that have not already been extracted. This avoids redundant unzip operations and keeps preprocessing focused on new data.

Function Descriptions

```
1. normalized_member_path(member_name: str) -> Path
```

This helper function normalizes file paths from ZIP member names by converting path separators to a consistent forward-slash format. The normalized value is returned as a `Path` object.

2. `is_zip_already_extracted(zip_path: Path, output_dir: Path) -> bool`

This helper function checks whether a ZIP archive appears to have already been extracted into the output directory. It reads all non-directory members from the archive and verifies that each expected file exists in the output location.

If the archive is invalid or empty, the function returns `False`.

3. `preprocess_surveys(input_dir: Path, output_dir: Path) -> bool`

This is the primary workflow function for preprocessing. It:

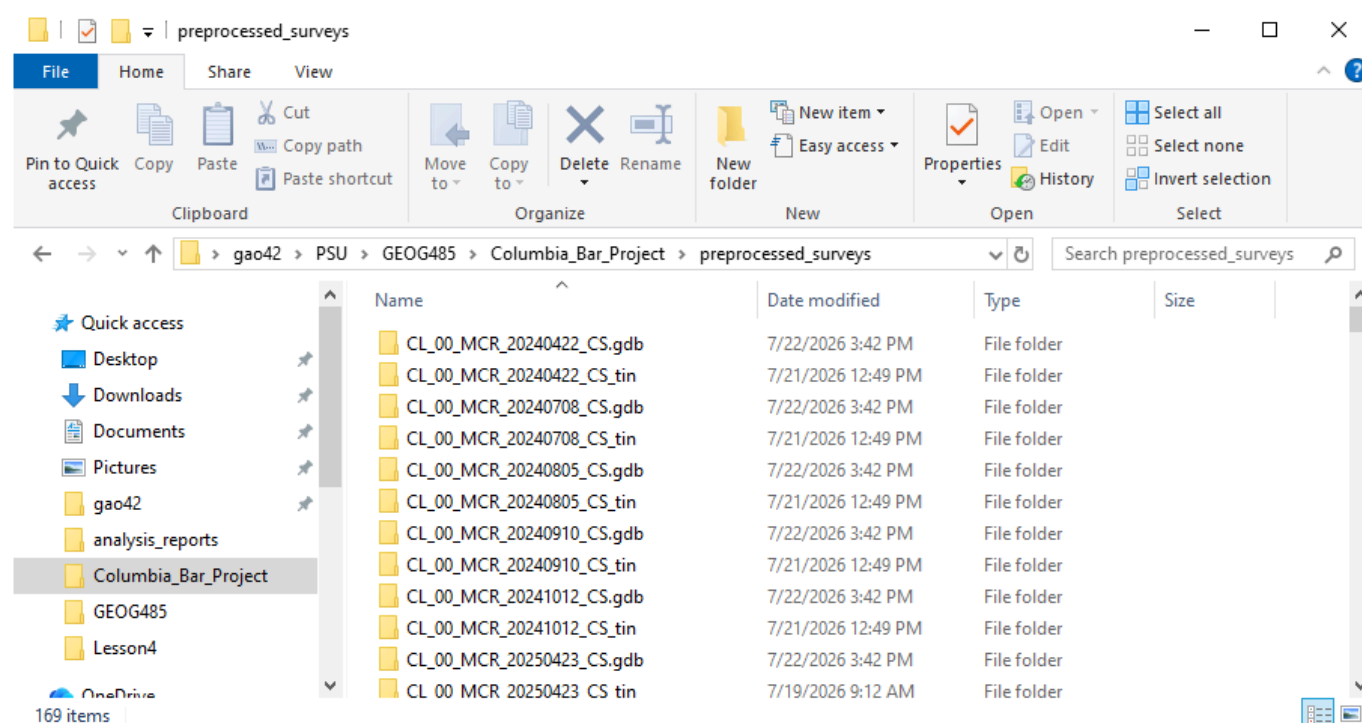
- validates that the input directory exists,
- creates the output directory if needed,
- scans for ZIP files in the input directory,
- skips archives already extracted,
- extracts new ZIP archives,
- logs skipped and extracted results,
- and prints a summary of preprocessing results.

The function returns `True` if at least one ZIP file was extracted and `False` otherwise.

4. `main() -> int`

This function sets preprocessing parameters, calls `preprocess_surveys(...)`, for unit testing & development and prints a final status message indicating whether new ZIP files were extracted.

Preprocessed Surveys Directory



Analyze Surveys

Purpose

The `analyze_surveys_v13.py` script performs bathymetric change analysis by comparing a latest survey raster with a prior survey raster selected by a month-based time window. It generates raw change, filtered change, deposition, and erosion raster outputs for each analysis run.

Function Descriptions

1. `add_months(dateVal, months) -> date`

This utility function shifts a date by a specified number of months and returns a valid calendar date. It is used to compute the target comparison date for selecting a prior survey.

2. `find_dated_gdbs(root_dir) -> list[tuple[date, str]]`

This utility function scans the survey source directory tree, finds file geodatabases (`.gdb`), extracts `YYYYMMDD` date tokens from folder names (or parent folder names), and returns a sorted list of dated geodatabase paths.

3. `analyze_surveys(months_diff, source_dir, output_dir, cell_size=5, threshold=0.25)`

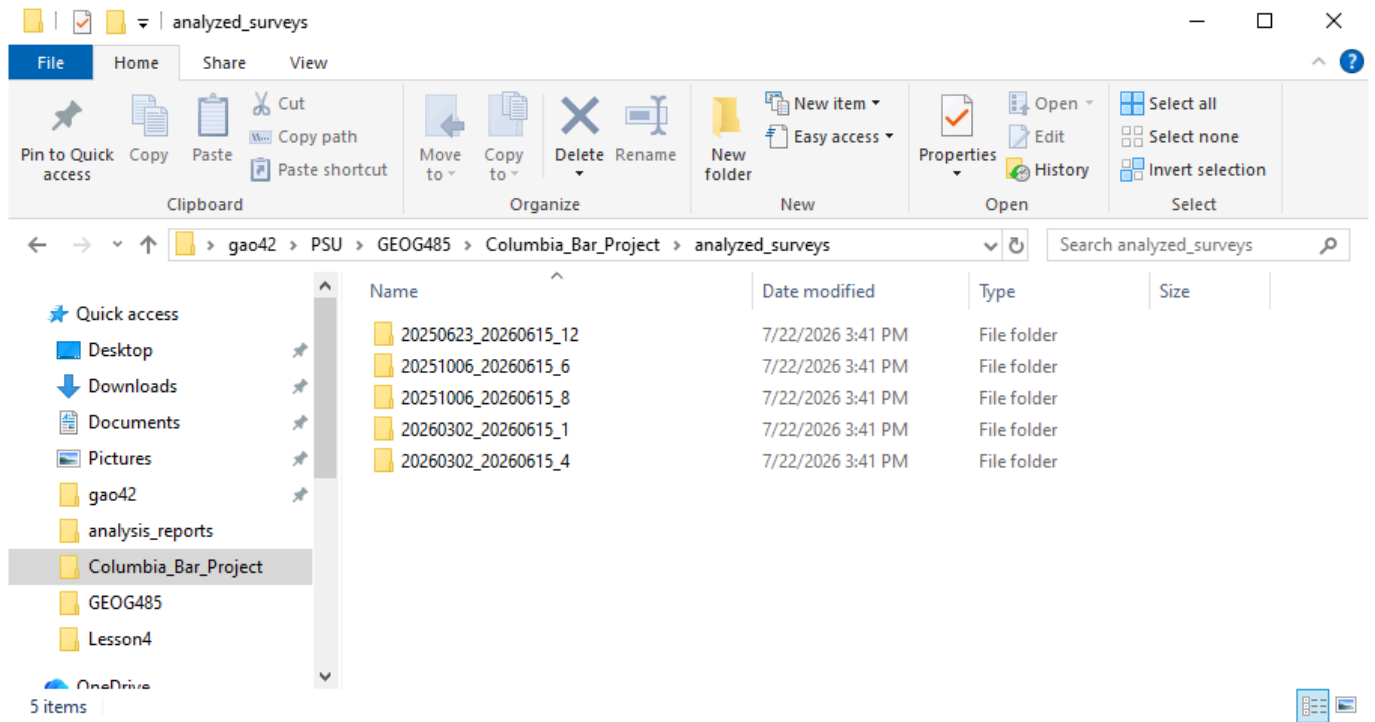
This is the primary analysis workflow function. It:

- validates source and output directories,
- identifies available dated survey geodatabases,
- selects the latest survey,
- computes a target date `months_diff` months in the past,
- selects the comparison survey nearest to that target date,
- validates that required `Bathymetry_Raster` inputs exist,
- creates a run-specific output folder,
- projects rasters to a common coordinate system,
- aligns raster grid properties,
- limits analysis to overlapping extent,
- computes the raw change raster,
- filters the change raster by threshold,
- creates deposition and erosion rasters,
- and saves all outputs.

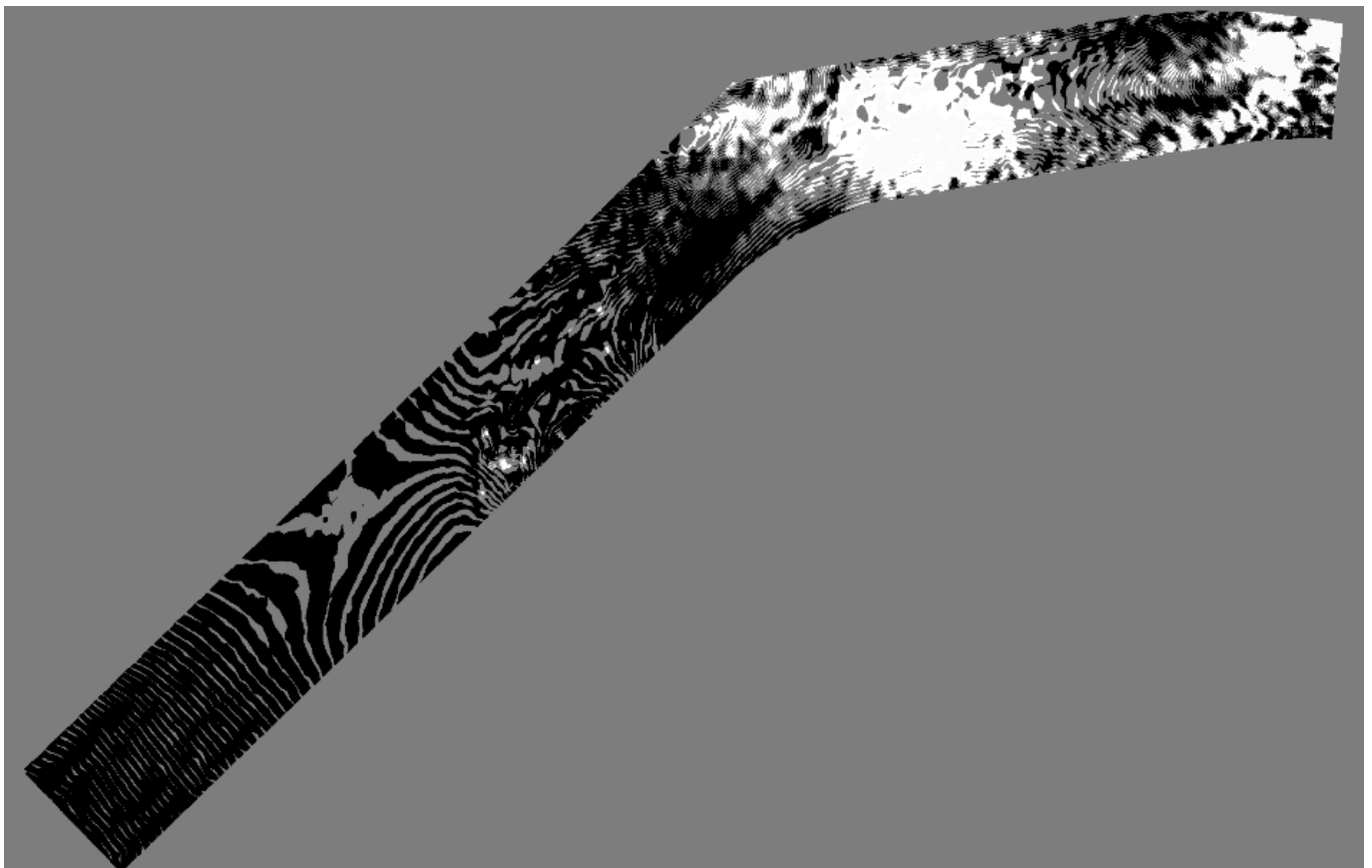
4. `main() -> int`

This function is the script entry point for unit testing and development. It sets parameter values (`source`, `output`, and `months_diff`), configures ArcPy environment settings, validates inputs, and calls `analyze_surveys(...)`.

Analyzed Surveys Directory



Bathimetric Difference Channel Analysis



Generate final reports

Purpose

The `generate_report_v5.py` script creates PDF map reports from completed analysis outputs. It scans the analyzed survey output directory, checks whether a report already exists for each analysis run, updates the

map layout text and extent, and exports only the missing reports.

Function Descriptions

1. `export_single_report(project_path, layout_name, raster_path, pdf_path, source_dir_name, zoom_out_factor=1.10, resolution=300) -> int`

This function creates one PDF report from a single filtered change raster. It:

- validates the ArcGIS Pro project and raster inputs,
- opens the project and locates the requested layout,
- updates the **Data** text element with the analysis run name,
- selects the map frame used for the report,
- removes any old raster layers,
- adds the current filtered raster,
- sets the map extent to the raster extent,
- applies an optional zoom-out factor,
- saves the project,
- and exports the layout to PDF.

The function returns **0** on success and **1** if a required input or layout element is missing.

2. `gen_report(analyzed_dir, reports_dir, project_path, layout_name, zoom_out_factor=1.10, resolution=300) -> int`

This function is the primary workflow for generating reports. It:

- creates the report output directory if needed,
- validates that the analyzed surveys directory exists,
- scans the analyzed directory for analysis run folders,
- builds the expected raster and report paths for each run,
- skips reports that already exist,
- skips runs where the filtered raster is missing,
- calls `export_single_report(...)` for each report that needs to be created,
- and prints a summary of generated, skipped, and failed reports.

The function returns **0** when the report generation process completes successfully and **1** if any report creation fails.

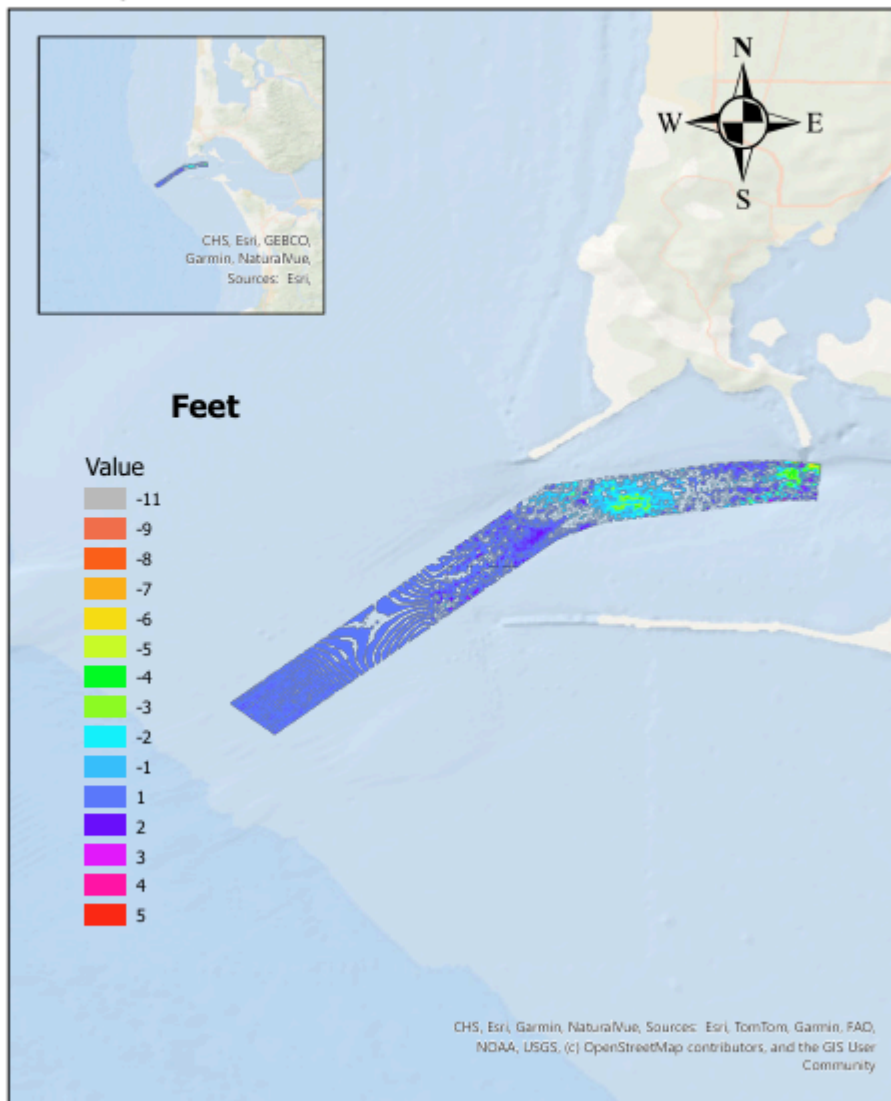
3. `main() -> int`

This function sets the report-generation parameters, defines the analyzed and report directories, and calls `gen_report(...)` for unit testing and development.

Generated Report

Columbia River Bar Channel Change Analysis

Survey Data Window: 20260302_20260615_1



Workflow Execution

Purpose

The `Columbia_Bar_Channel_Analysis_v4.py` script orchestrates the full Columbia River Bar channel workflow by calling the standalone module functions in sequence. It serves as the top-level execution script for end-to-end processing.

Execution Structure

This script is organized as a linear workflow with parameter definitions followed by four execution steps:

1. download new surveys,
2. preprocess downloaded surveys,

3. run analysis for multiple time windows,
4. generate PDF reports.

The script uses comments to label each stage and to describe the intent of each block.

Function Calls Used in the Execution Script

1. `check_and_download_surveys(...)` from `download_new_surveys_v4.py`

This call queries the USACE source, applies date and text filters, downloads new surveys, and returns a boolean indicating whether new downloads occurred.

2. `preprocess_surveys(...)` from `preprocess_surveys_v2.py`

This call unzips newly downloaded surveys and skips archives that have already been extracted.

3. `analyze_surveys(...)` from `analyze_surveys_v13.py`

This call performs the raster change analysis. In this execution script, it is run three times with different `months_diff` values (1, 6, and 12) to generate multiple comparison windows.

4. `gen_report(...)` from `generate_report_v5.py`

This call scans analysis outputs and generates any missing report PDFs in the report output directory.

Notes

- `Columbia_Bar_Channel_Analysis_v4.py` is an execution script and does not define local workflow functions.
- The script-level comments and parameter block provide the configuration context for all module calls.
- It should also be noted that some of the analysis have truncated channels do to the survey source raster files.

Truncated Source Rasters

